



SECP3843-01

TUTORIAL 2

LECTURER : DR ARYATI BINTI BAKRI

NO	NAME	MATRIC NO
1.	PRAVINRAJ A/L SIVABATHI	A23CS0171
2.	DHESHIEGHAN A/L SARAVANA MOORTHY	A23CS0072
3.	AHMAD ZIYAAD BIN MOHD ABBAS	A23CS0206

1.0 Business Understanding

1.1 Problem Statement

Education is one of the most critical pillars of national development, and understanding the state of basic education across a large and diverse country such as Brazil requires sophisticated data management and analytical capabilities. The Brazilian National Basic Education Census (Censo Escolar) is one of the largest educational data collection efforts in the world, gathering detailed information annually from millions of schools, students, teachers, and facilities across all 26 states and the Federal District of Brazil. However, the sheer volume and complexity of this data spanning from 2010 to 2020 and containing hundreds of attributes per record makes traditional data processing approaches inadequate for timely and meaningful analysis.

The core business problem addressed in this tutorial is the need to build a scalable and efficient Extract, Transform, Load (ETL) data pipeline that can ingest raw CSV data from the Brazilian government's open data portal, process and transform it into an analytically optimised Star Schema structure, and store it in a relational database ready for business intelligence (BI) and dashboard reporting. Without such a pipeline, stakeholders such as education policymakers, school administrators, and researchers would struggle to derive timely and accurate insights from the data, potentially leading to poorly informed decisions about resource allocation, infrastructure investment, and educational reform.

1.2 Business Objectives

The primary business objective of this project is to transform approximately 2.2 GB of raw annual CSV files covering the Brazilian Basic Education Census from 2010 to 2020 into a structured, query-optimised data warehouse using Apache Spark as the core processing engine. The specific objectives are to design and implement a pipeline that extracts raw CSV data automatically from the Brazilian Open Data portal, converts these files into the Parquet format for improved performance, transforms the data into a Star Schema consisting of clearly defined dimension and fact tables, and loads the final structured data into a PostgreSQL relational database. A secondary objective is to enable business intelligence dashboards through Metabase, which connects to the PostgreSQL database and allows stakeholders to visualise trends in school enrolments across years, states, and cities.

1.3 Stakeholders and Context

The primary stakeholders of this system include the Brazilian Ministry of Education (MEC) and the National Institute for Educational Studies and Research (INEP), which are responsible for collecting and publishing Censo Escolar data annually. Secondary stakeholders include state and municipal education secretariats that rely on this data to plan school infrastructure, allocate teachers, and manage

student enrolment. Policy researchers and data analysts who require aggregated and clean datasets for longitudinal studies also stand to benefit significantly from this pipeline. By building an automated and reproducible ETL pipeline, this project directly supports these stakeholders in their goal of making data-driven decisions to improve educational outcomes across Brazil.

2.0 Data Understanding

2.1 Dataset Overview

The dataset is sourced from the Brazilian National Basic Education Census (Censo Escolar), published annually by INEP and freely available at <https://censos.inep.gov.br/>. The dataset spans 2010 to 2020, producing eleven annual CSV files. Each file contains approximately 370 columns and around 180 MB of disk space, bringing the total raw dataset to approximately 2.2 GB. The data captures every registered basic education school in Brazil, including attributes related to school infrastructure, geographical location, student demographics, teacher qualifications, and availability of resources such as internet access, libraries, water supply, sanitation, and computer laboratories.

2.2 Data Attributes and Types

The Censo Escolar dataset contains a rich variety of attributes across several categories as shown in the table below

Attribute	Category	Data Type	Description
CO_UF	Geographical	Integer	Unique numeric code identifying the state
NO_UF	Geographical	String	Full name of the state
SG_UF	Geographical	String	Two-letter abbreviation of the state
CO_MUNICIPIO	Geographical	Integer	Unique numeric code identifying the municipality
NO_MUNICIPIO	Geographical	String	Full name of the municipality
TP_DEPENDENCIA	Administrative	Integer	School dependency type (1=Federal, 2=State, 3=Municipal, 4=Private)
TP_LOCALIZACAO	Administrative	Integer	School location type (1=Urban, 2=Rural)
IN_AGUA_POTAVEL	Infrastructure	Binary (0/1)	Indicates whether the school has access to potable water

IN_BANHEIRO	Infrastructure	Binary (0/1)	Indicates whether the school has bathroom facilities
IN_BIBLIOTECA	Infrastructure	Binary (0/1)	Indicates whether the school has a library
IN_COMPUTADOR	Infrastructure	Binary (0/1)	Indicates whether the school has computers
IN_ENERGIA_INEXISTENTE	Infrastructure	Binary (0/1)	Indicates whether the school has no electricity supply
IN_ESGOTO_INEXISTENTE	Infrastructure	Binary (0/1)	Indicates whether the school has no sewage system
IN_INTERNET	Infrastructure	Binary (0/1)	Indicates whether the school has internet access
IN_REFEITORIO	Infrastructure	Binary (0/1)	Indicates whether the school has a canteen or dining facility
NU_ANO_CENSO	Temporal	Integer	The census year in which the record was collected (2010–2020)
QT_MAT_BAS	Enrolment Metric	Integer	Total number of basic education enrolments per school per year

2.3 Data Quality and Issues

Upon initial exploration of the raw CSV files, several data quality issues were identified. The files use a semicolon (;) as the field delimiter rather than a comma, and they are encoded in Latin-1 (ISO-8859-1) rather than UTF-8, which are common characteristics of Brazilian government datasets requiring explicit handling during ingestion. Certain fields also contain missing or null values, particularly in infrastructure indicator columns where schools may not have reported specific attributes. The inferSchema feature in PySpark was used to automatically detect column data types, though this requires an additional full scan of the data. Given the volume of over 2.7 million final fact records, efficient handling of these issues was essential to maintain pipeline performance.

2.4 Relational Data Model

The relational data model reflects the structure of the raw Censo Escolar data before transformation into the analytical Star Schema. At the centre is the school entity, identified by a unique school code, with relationships to its geographical location, administrative classification, and annual enrolment and infrastructure records. The location entity captures the hierarchical relationship between

municipalities and states, with each school belonging to exactly one municipality and each municipality belonging to exactly one state. The dependency type entity classifies schools by administrative ownership, and the location type entity distinguishes between urban and rural schools. Each school also has multiple annual records capturing infrastructure indicators and enrolment figures per census year, connected through foreign key references to ensure referential integrity.

3.0 Data Preparation

Job Id ▾	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
10	jdbc at NativeMethodAccessorImpl.java:0 jdbc at NativeMethodAccessorImpl.java:0	2026/05/16 20:31:54	0.1 s	1/1 (1 skipped)	1/1 (4 skipped)
9	jdbc at NativeMethodAccessorImpl.java:0 jdbc at NativeMethodAccessorImpl.java:0	2026/05/16 20:31:53	0.7 s	1/1	4/4
8	jdbc at NativeMethodAccessorImpl.java:0 jdbc at NativeMethodAccessorImpl.java:0	2026/05/16 20:31:49	87 ms	1/1 (1 skipped)	1/1 (4 skipped)
7	jdbc at NativeMethodAccessorImpl.java:0 jdbc at NativeMethodAccessorImpl.java:0	2026/05/16 20:31:49	0.5 s	1/1	4/4
6	jdbc at NativeMethodAccessorImpl.java:0 jdbc at NativeMethodAccessorImpl.java:0	2026/05/16 20:31:45	0.2 s	1/1 (1 skipped)	1/1 (4 skipped)
5	jdbc at NativeMethodAccessorImpl.java:0 jdbc at NativeMethodAccessorImpl.java:0	2026/05/16 20:31:44	0.7 s	1/1	4/4
4	jdbc at NativeMethodAccessorImpl.java:0 jdbc at NativeMethodAccessorImpl.java:0	2026/05/16 20:31:37	2 s	1/1 (1 skipped)	1/1 (4 skipped)
3	jdbc at NativeMethodAccessorImpl.java:0 jdbc at NativeMethodAccessorImpl.java:0	2026/05/16 20:31:32	4 s	1/1	4/4
2	count at NativeMethodAccessorImpl.java:0 count at NativeMethodAccessorImpl.java:0	2026/05/16 20:30:27	0.3 s	1/1 (1 skipped)	1/1 (4 skipped)
1	count at NativeMethodAccessorImpl.java:0 count at NativeMethodAccessorImpl.java:0	2026/05/16 20:30:25	2 s	1/1	4/4
0	parquet at NativeMethodAccessorImpl.java:0 parquet at NativeMethodAccessorImpl.java:0	2026/05/16 20:30:20	1 s	1/1	1/1

Figure 1: Apache Spark UI showing 11 successfully completed jobs during the ETL pipeline execution

3.1 Data Extraction

The first step involved extracting raw CSV files from the Brazilian Open Data portal using a Python script that invokes Linux shell commands through the `os` library to download eleven annual files covering 2010 to 2020. Each file was stored locally in a dedicated data directory. This approach ensures reproducibility, as the download script can be re-executed to refresh the dataset whenever new Censo Escolar data is published. The downloaded files are named consistently by year, allowing subsequent processing steps to iterate over them in a predictable sequence.

3.2 CSV to Parquet Conversion

Once the raw CSV files were downloaded, the next step was converting them from CSV to Apache Parquet format using PySpark. This conversion is a critical performance optimization because Parquet is a columnar storage format significantly more compact and faster to query than CSV, particularly for analytical workloads accessing only a subset of columns. A `SparkSession` was initialised with the application name `CensoEscolarStarSchema` as the entry point for all Spark operations. Each CSV file

was read using Spark's DataFrameReader with the header option set to true, the delimiter set to semicolon, encoding set to Latin-1, and inferSchema enabled to automatically detect column data types. The resulting DataFrame was written to disk in Parquet format using overwrite mode, producing files approximately five times smaller than the original CSVs. The Parquet files were subsequently reloaded into Spark for the transformation phase.

3.3 Handling Missing Values and Inconsistencies

Missing values in infrastructure indicator columns were handled by treating null entries as 0, indicating the absence of a facility or resource, consistent with the binary semantics of these variables where a null value most likely reflects non-reporting rather than actual presence of the facility. Column data types were cast explicitly where necessary to ensure consistency across years, since the inferred schema from one year may differ slightly from another. String columns containing state and municipality names were retained as-is, while numeric columns such as school codes and enrolment counts were cast to integer types. These preprocessing steps ensure that the data loaded into the Star Schema is clean, consistent, and analytically ready.

4.0 Modeling

4.1 Multidimensional Data Model Star Schema

	Table	Engine	Data Length?	Index Length?	Data Free	Auto Increment	Rows?	Comment?
☐	dim_in_agua_potavel	table	16,384	0	?	?	?	
☐	dim_in_banheiro	table	16,384	0	?	?	?	
☐	dim_in_biblioteca	table	16,384	0	?	?	?	
☐	dim_in_computador	table	16,384	0	?	?	?	
☐	dim_in_energia_inexistente	table	16,384	0	?	?	?	
☐	dim_in_equip_nenhum	table	16,384	0	?	?	?	
☐	dim_in_esgoto_inexistente	table	16,384	0	?	?	?	
☐	dim_in_internet	table	16,384	0	?	?	?	
☐	dim_in_refeitorio	table	16,384	0	?	?	?	
☐	dim_local	table	483,328	0	?	?	~ 5,570	
☐	dim_tp_dependencia	table	16,384	0	?	?	?	
☐	dim_tp_localizacao	table	16,384	0	?	?	?	
☐	fact_censo_escolar	table	532,267,008	0	?	?	~ 2,792,940	
☐	test_products	table	16,384	0	?	?	?	
	14 in total		en_US.utf8	532,946,944	0	0		

Figure 2: PostgreSQL database showing all Star Schema tables (10 dimension tables + 1 fact table) created successfully

The modeling phase centred on designing and implementing a multidimensional Star Schema, the industry-standard data architecture for analytical and business intelligence workloads. The Star Schema organises data into a central fact table surrounded by multiple dimension tables, each connected via foreign key references. This structure significantly accelerates aggregation queries by reducing complex joins during reporting, which is particularly beneficial when querying nearly three million records.

The Star Schema implemented in this project consists of one fact table and ten dimension tables. The fact table, named fact_censo_escolar, contains the total number of enrolments per school per year along with foreign key references to each dimension table. The dimension tables capture descriptive

attributes and include `dim_local` for geographical information about municipality and state, `dim_tp_dependencia` for administrative dependency type, and `dim_tp_localizacao` for urban or rural classification. The remaining eight dimension tables — `dim_in_agua_potavel`, `dim_in_banheiro`, `dim_in_biblioteca`, `dim_in_computador`, `dim_in_energia_inexistente`, `dim_in_esgoto_inexistente`, `dim_in_internet`, and `dim_in_refeitorio` — correspond to the binary infrastructure indicators, each storing the distinct value combinations with a surrogate primary key referenced by the fact table.

4.2 Implementation with Apache Spark and PostgreSQL

The Star Schema was implemented using PySpark for data transformation and `psycopg2` for database interaction with PostgreSQL. For each dimension table, relevant columns were selected from the Parquet DataFrame, duplicates removed using Spark's `distinct` transformation, and a unique surrogate key assigned using `monotonically_increasing_id`. The resulting dimension DataFrames were written to PostgreSQL using Spark's JDBC connector with `overwrite` mode, which automatically creates or replaces the corresponding tables. Primary keys were subsequently defined using `psycopg2` to execute `ALTER TABLE` SQL statements directly against PostgreSQL. The fact table was constructed through a series of left join operations between the main DataFrame and each dimension table, matching on natural key columns. After each join, natural key columns were dropped and replaced by the dimension surrogate foreign key. The final fact DataFrame was written to PostgreSQL using `append` mode, preserving historical data and allowing incremental pipeline runs. The complete pipeline produced a fact table containing approximately 2,792,940 records, confirming successful end-to-end execution.

4.3 Pipeline Architecture

☐		Name	Container ID	Image	Port(s)	CPU (%)	Last	Actions
☐	▼	spark_etl_proje	-	-	-	1.98%	3 ho	   
☐		spark_postgr	0867f264ae5c	postgres:1	5432:5432 	0%	19 h	   
☐		metabase_ce	551d50091a46	metabase/	3000:3000 	1.98%	19 h	   
☐		adminer_spa	47963b5a023a	adminer	8081:8080 	0%	3 ho	   

Figure 3: Docker Desktop showing all 4 containers running Spark ETL, PostgreSQL, Metabase, and Adminer

The pipeline was orchestrated using Apache Spark version 3.5.1 in a local environment managed by Docker. The Docker Compose configuration brought up four containers: the Spark ETL pipeline container, a PostgreSQL container serving as the data warehouse, a Metabase container for dashboard visualisation, and an Adminer container for database administration. The Apache Spark UI confirmed 11 Spark jobs were completed successfully, with job durations ranging from 87 milliseconds to 4 seconds, demonstrating the efficiency of Spark's lazy evaluation model in optimising execution plans before processing data.

5.0 Evaluation

5.1 Pipeline Performance

The evaluation focused on both technical correctness and business utility. From a technical perspective, the pipeline successfully completed all 11 Spark jobs as evidenced by the Spark UI showing every job at 100% task completion. The CSV to Parquet conversion produced approximately 80% file size reduction, confirming the expected storage efficiency gains. The fact table containing approximately 2,792,940 records was populated correctly, with all dimension foreign keys resolving without null references, confirming Star Schema joint integrity.

5.2 Business Utility and Dashboard Results

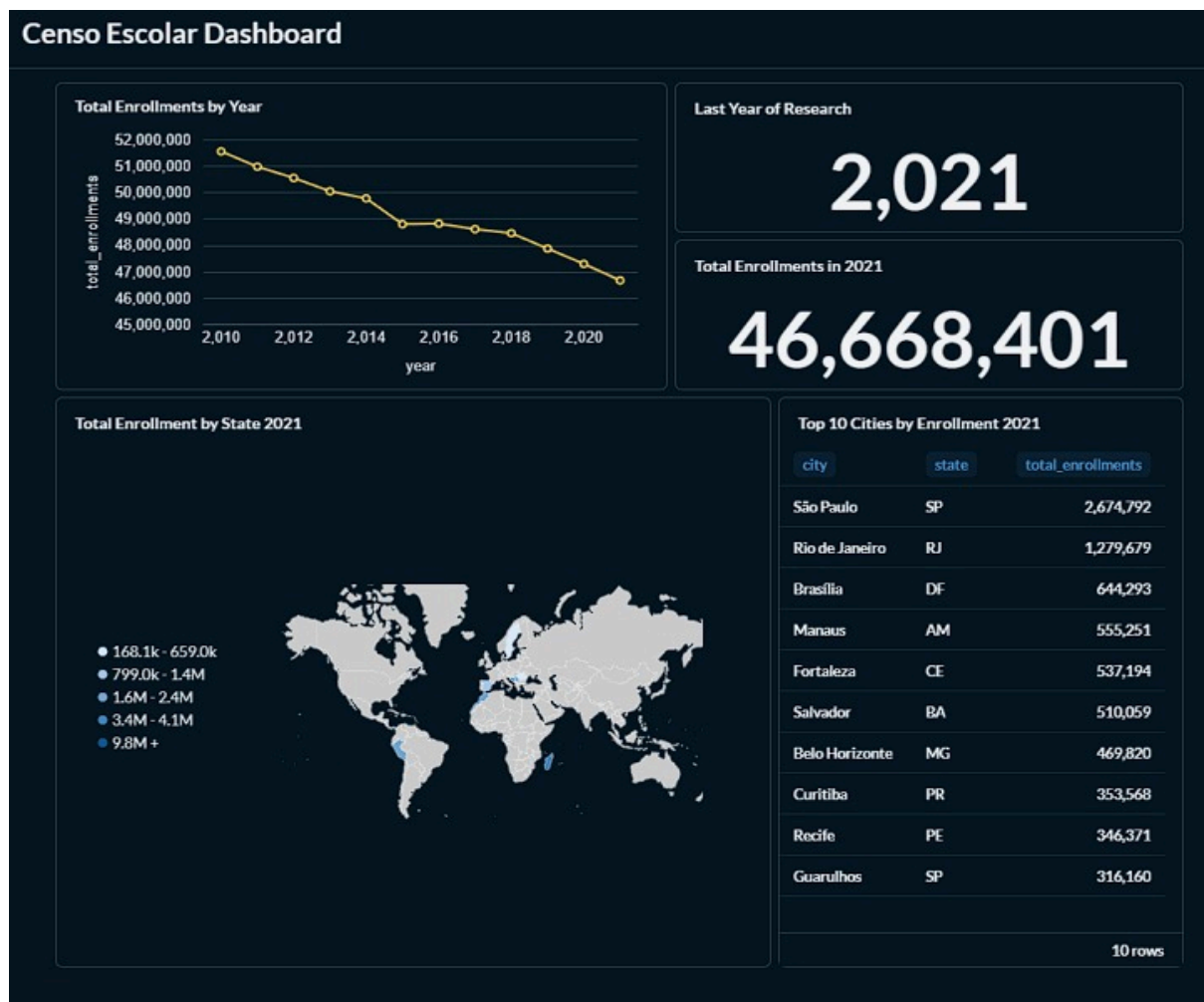


Figure 4: Metabase dashboard showing total enrolments by year and top 10 cities by enrolment in 2021

From a business utility perspective, the Star Schema enabled the construction of a Metabase dashboard answering key analytical questions about Brazilian basic education enrolments. The dashboard clearly shows that total enrolments peaked at approximately 52 million in 2010 and declined steadily to around 46.7 million by 2021, reflecting demographic and policy changes in Brazil's education system over the decade. The geographical breakdown in 2021 highlights São Paulo as the most populous state for enrolments with approximately 2,674,792 students, followed by Rio de

Janeiro with 1,279,679 and Brasília with 644,293. The top-10 cities table provides granular visibility into urban centres with the highest student concentrations, which is directly actionable for resource planning and teacher allocation decisions.

5.3 Limitations and Future Improvements

Despite successful execution, several limitations were identified. First, the use of inferSchema during CSV loading requires a full file scan to detect data types, adding processing time for large datasets a predefined schema would improve performance and reduce type inference errors. Second, the current batch processing approach requires reloading the entire dataset if a correction is made to a single year, whereas incremental loading with year-based partitioning would significantly improve efficiency. Third, the Metabase dashboards are currently limited to enrolment counts and geographical breakdowns. Incorporating infrastructure quality metrics such as the proportion of schools with internet access or clean water would provide richer insights for policymakers. Future work could also explore integrating machine learning models to predict future enrolment trends based on demographic and socioeconomic variables.

6.0 Reflection

PRAVINRAJ A/L SIVABATHI

Throughout the execution of this tutorial, I encountered several technical challenges that significantly contributed to my growth as a data engineer. The most demanding aspect was configuring the Docker environment, particularly setting up the network connections between the Spark, PostgreSQL, Metabase, and Adminer containers. Understanding how each container communicates with the others through Docker Compose required careful reading of the official documentation and multiple rounds of debugging before the pipeline ran successfully end-to-end. In terms of skills gained, this project gave me hands-on experience with PySpark that I had never had before, especially in reading and writing Parquet files and performing complex DataFrame transformations such as distinct, join, and monotonically_increasing_id for surrogate key generation. I also developed a much deeper understanding of the Star Schema design, particularly why dimension tables are separated from the fact table and how this separation dramatically speeds up aggregation queries in business intelligence tools like Metabase. From a teamwork perspective, I learned the importance of clear communication when dividing pipeline responsibilities, as each stage of the ETL process depends on the output of the previous stage. Working collaboratively and reviewing each other's code ensured that errors were caught early and that every team member had a solid understanding of the complete pipeline rather than just their individual component.

DHESHIEGHAN A/L SARAVANA MOORTHY

My experience working on this ETL pipeline tutorial was both challenging and highly rewarding from a personal learning perspective. One of the biggest challenges I faced was understanding the JDBC connection between Apache Spark and PostgreSQL, specifically identifying the correct PostgreSQL JAR driver version and configuring it within the SparkSession builder so that Spark could write DataFrames directly into the database. This required extensive troubleshooting and cross-referencing between the Spark documentation and the PostgreSQL connector guides before the connection was finally established successfully. In terms of technical skills, I gained significant proficiency in multidimensional data modeling, particularly in understanding how to design and populate a Star Schema with multiple dimension tables and a central fact table. Implementing the join operations

between the main DataFrame and each dimension table to resolve surrogate foreign keys was a particularly valuable exercise that deepened my understanding of how data warehouses are structured in real-world data engineering projects. From a teamwork and collaboration standpoint, this project reinforced for me how important it is to maintain shared ownership of a codebase even when responsibilities are divided. Regular team check-ins and code walkthroughs meant that I was never isolated in my work, and the collaborative debugging sessions helped us resolve problems much faster than any of us could have done individually, ultimately resulting in a pipeline that successfully processed nearly three million records end-to-end.

AHMAD ZIYAAD BIN MOHD ABBAS

Completing this instructional module provided me with valuable insights into building Extract Transform Load workflows utilizing Apache Spark. I gained practical experience with the entire lifecycle, which involves pulling initial raw datasets, cleaning and formatting them into a highly organized structure, and eventually migrating the finalized information into an appropriate storage solution. The assignment made it clear that successful data engineering requires much more than mere programming skills. It fundamentally relies on architecting robust workflows, ensuring the reliability of the information, and arranging it for analytical tasks utilizing relational and multidimensional frameworks. Additionally, I recognized the immense value of Apache Spark for executing rapid operations, particularly when dealing with massive volumes of information. During the exercise, I encountered a few obstacles. Specifically, I initially struggled to map individual Spark functions to their corresponding phases within the overarching data lifecycle. It was also difficult to align these technical procedures with the CRISP DM reporting framework. Nevertheless, by thoroughly revisiting the educational materials, I managed to gain a much deeper clarity regarding the exact goal of every individual phase. In conclusion, this learning experience significantly broadened my knowledge of Apache Spark, the fundamentals of data preparation, and the critical role that comprehensive record keeping plays within technical engineering initiatives.